Sumário

1 4ª Entrega: Projeto de Banco de Dados **2**
1.1 Escolha do SGBD . . . 2
1.2 Melhorias em relação ao diagrama de classes . . . 2
1.3 Mapeamento classes para tabelas . . . 4
1.4 Modelo lógico . . . 4
1.4.1 Esquemas relacionais . . . 4
1.4.2 Normalização . . . 5
1.5 Modelo físico . . . 5
1.5.1 Rastreabilidade das regras de negócio . . . 7
1.6 Dicionário de dados . . . 7
1.6.1 Tabela `usuario` . . . 7
1.6.2 Tabela `cliente` . . . 8
1.6.3 Tabela `endereco` . . . 8
1.6.4 Tabela `categoria` . . . 8
1.6.5 Tabela `cupcake` . . . 9
1.6.6 Tabela `cupcake_categoria` . . . 9
1.6.7 Tabela `carrinho` . . . 9
1.6.8 Tabela `item_carrinho` . . . 9
1.6.9 Tabela `pedido` . . . 10
1.6.10 Tabela `item_pedido` . . . 10
1.6.11 Tabela `pagamento` . . . 10
1.6.12 Tabela `transicao_status_pedido` . . . 11
1.6.13 Tabela `historico_status_pedido` . . . 11
1.6.14 Tipos enumerados . . . 11
1.6.15 Fluxo permitido de status do pedido (RN14) . . . 11
1.7 Verificação do projeto físico . . . 12

2 5ª Entrega: Implementação do Sistema **13**
2.1 Repositório . . . 13
2.2 Tecnologias . . . 13
2.3 Arquitetura MVC . . . 14
2.4 Telas e navegação . . . 14
2.5 Interface de programação (API REST) . . . 14
2.6 Autenticação e segurança . . . 15
2.7 Rastreabilidade: histórias de usuário e implementação . . . 15
2.8 Testes automatizados . . . 16
2.9 Simplificações e trabalhos futuros . . . 16

A Script SQL completo (PostgreSQL) **17**

1 4ª Entrega: Projeto de Banco de Dados

Esta seção refina o diagrama de classes da 2ª entrega e o transforma em um projeto de dados completo: modelo lógico normalizado, projeto físico em SGBD relacional e dicionário de dados. O script executável está no arquivo `schema_loja_cupcakes.sql` (Apêndice A).

Os diagramas UML revisados e o projeto conceitual de dados, que antecedem este projeto físico, estão no documento *Revisão da Modelagem UML e Projeto Conceitual de Dados*.

Repositório: o script SQL, o código do sistema e esta documentação estão publicados em `https://github.com/Fellp-coffww/Loja-Cupcake`. O script está em `sql/schema_loja_cupca`

1.1 Escolha do SGBD

Foi adotado o **PostgreSQL**, SGBD relacional de código aberto, amplamente usado no mercado e de fácil manutenção. Ele oferece tipos enumerados, colunas geradas, índices parciais, *triggers* e transações ACID, recursos usados neste projeto para garantir as regras de negócio (RN01 a RN15) diretamente no banco.

1.2 Melhorias em relação ao diagrama de classes

O diagrama de classes descreve objetos e comportamentos. Para virar um banco relacional íntegro, foram feitos os ajustes abaixo.

Ponto	Diagrama original	Projeto de banco
Endereço	Atributo `endereco: String` em Cliente	Tabela `endereco` (CEP, rua, número etc.), atendendo à validação de campos da US07
Categoria	Um cupcake tem uma categoria	Relação N:N (`cupcake_categoria`): um cupcake pode ser "Chocolate" e "Vegano" (US02)
Itens do pedido	Não existe	Tabela `item_pedido`, com preço unitário gravado no momento da compra
Pagamento	Relação 1..1 com Pedido	Relação 1:N: o CU10 prevê pagamento recusado e nova tentativa, então um pedido pode ter várias tentativas
Status do pedido	`status: String`	Tipo enumerado com fluxo validado (RN14) e tabela de histórico (US11)
Valores monetários	`double`	`NUMERIC(10,2)`, evitando erros de arredondamento
Carrinho	Lista em memória	Tabelas `carrinho` e `item_carrinho` persistidas (RNF04)
Relatório	Classe	Não vira tabela: é gerado por *views* (dashboard da US15 e estoque)
Administrador	Subclasse de Usuário	Representado pelo campo `perfil` de `usuario`; não tem atributos próprios
Estoque	Citado em `gerarEstoque()`, sem atributo	Coluna `cupcake.estoque`, com baixa automática ao confirmar o pedido

Tabela 1: Ajustes feitos do diagrama de classes para o banco de dados

1.3 Mapeamento classes para tabelas

Classe	Tabela	Estratégia
Usuario, Administrador	`usuario`	Hierarquia em uma tabela, com discriminador `perfil`
Cliente	`cliente`, `endereco`	Especialização 1:1; endereço normalizado
Cupcake	`cupcake`	Mapeamento direto
Categoria	`categoria`, `cupcake_categoria`	Associação N:N
Carrinho, ItemCarrinho	`carrinho`, `item_carrinho`	Agregação 1:N; chave composta nos itens
Pedido	`pedido`, `item_pedido`, `historico_status_pedido`	Agregação 1:N e histórico
Pagamento	`pagamento`	Associação 1:N com Pedido
Relatorio	*views*	Consultas derivadas
—	`transicao_status_pedido`	Apoio para validar o fluxo de status

Tabela 2: Mapeamento do modelo orientado a objetos para o relacional

1.4 Modelo lógico

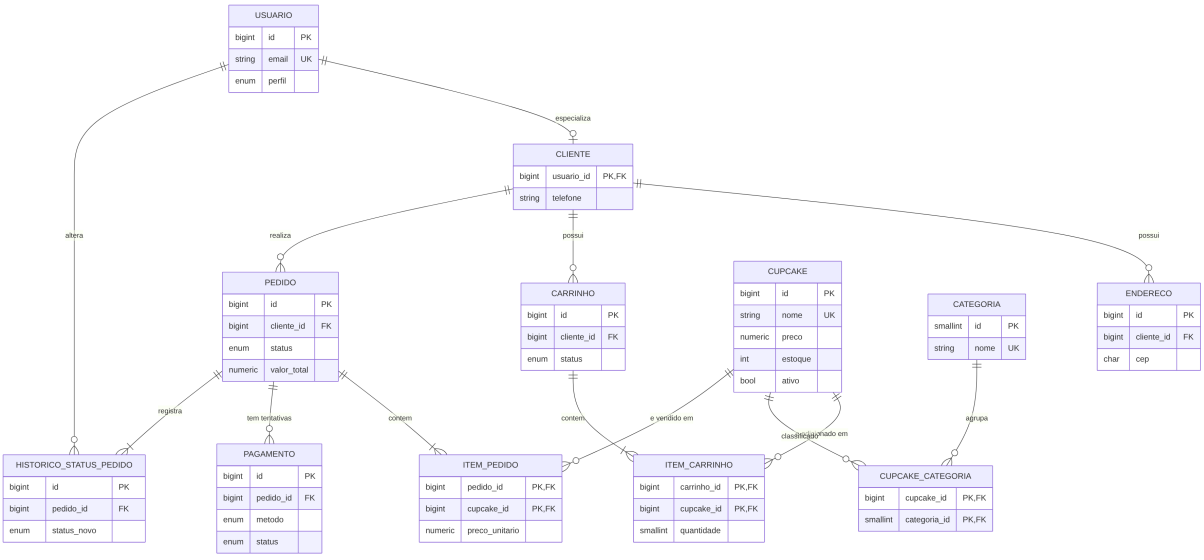


Figura 1: Modelo lógico relacional (entidades, chaves e cardinalidades)

1.4.1 Esquemas relacionais

Notação: sublinhado indica chave primária e → indica chave estrangeira.

- **usuario**(id, nome, email, senha_hash, perfil, ativo, criado_em)
- **cliente**(usuario_id → usuario, telefone)

- **endereco**(id, cliente_id → cliente, apelido, cep, logradouro, numero, complemento, bairro, cidade, uf, principal, ativo)
- **categoria**(id, nome, ativa)
- **cupcake**(id, nome, descricao, ingredientes, preco, imagem_url, estoque, destaque, ativo, criado_em, atualizado_em)
- **cupcake_categoria**(cupcake_id → cupcake, categoria_id → categoria)
- **carrinho**(id, cliente_id → cliente, status, criado_em, atualizado_em)
- **item_carrinho**(carrinho_id → carrinho, cupcake_id → cupcake, quantidade)
- **pedido**(id, cliente_id → cliente, status, data_pedido, entrega_cep, entrega_logradouro, entrega_numero, entrega_complemento, entrega_bairro, entrega_cidade, entrega_uf, taxa_entrega, valor_total, atualizado_em)
- **item_pedido**(pedido_id → pedido, cupcake_id → cupcake, quantidade, preco_unitario, subtotal)
- **pagamento**(id, pedido_id → pedido, metodo, status, valor, gateway_transacao_id, criado_em, processado_em)
- **transicao_status_pedido**(de, para)
- **historico_status_pedido**(id, pedido_id → pedido, status_anterior, status_novo, alterado_por → usuario, alterado_em)

1.4.2 Normalização

- **1FN:** todos os atributos são atômicos. O endereço foi decomposto em campos, e a lista `itens` da classe Carrinho virou uma tabela própria.
- **2FN:** nas tabelas de chave composta (`item_carrinho`, `item_pedido`, `cupcake_categoria`), os atributos dependem da chave inteira. O nome e o preço atual do produto ficam apenas em `cupcake`.
- **3FN:** não há dependências transitivas. Categoria, cliente e produto são referenciados por chave, e não copiados.
- **Desnormalizações intencionais:** `item_pedido.preco_unitario` e as colunas `entrega_*` de `pedido` guardam o valor vigente na compra. Isso preserva o histórico mesmo que o preço ou o endereço mudem depois. `valor_total` e `subtotal` são conferidos por regra de negócio.

1.5 Modelo físico

O script completo está no Apêndice A. Os principais elementos são:

- **Esquema** `loja`, com 13 tabelas.
- **Tipos e domínios:** `status_pedido`, `status_pagamento`, `metodo_pagamento`, `perfil_usuari` `status_carrinho`, `d_cep`, `d_uf` e `d_dinheiro`.
- **Chaves primárias** do tipo `IDENTITY` e chaves compostas nas tabelas associativas.

- **Índices únicos parciais:** um carrinho aberto por cliente, um endereço principal por cliente e um pagamento aprovado por pedido.
- **Índices de desempenho** para a vitrine (RNF01), filtro por categoria (RNF02), pedidos por cliente e relatórios (RNF15).
- **Triggers:** validação do fluxo de status (RN14), histórico automático, aprovação de pagamento que muda o pedido para PAGO, e atualização de `atualizado_em`.
- **Função** `fn_confirmar_pedido`: transforma o carrinho em pedido em uma única transação (copia preços e endereço, baixa o estoque e fecha o carrinho).
- **Views:** `vw_vitrine`, `vw_carrinho_detalhado`, `vw_vendas_diarias`, `vw_produtos_mais_ven` `vw_estoque` e `vw_pedidos_resumo`.
- **Segurança:** o papel `app_loja` não pode apagar pedidos, itens, pagamentos nem histórico. Senhas são armazenadas somente como *hash* (SHA-256 na versão do trabalho), e dados de cartão nunca são guardados (RNF08).

1.5.1 Rastreabilidade das regras de negócio

Regra	Descrição	Mecanismo no banco
RN01	Exibir apenas produtos ativos	Coluna `cupcake.ativo`, filtro em `vw_vitrine` e índice parcial
RN02	Categoria deve existir	FK de `cupcake_categoria` para `categoria`
RN03	Dados atualizados	Leitura direta da tabela; `atualizado_em` via trigger
RN04	Quantidade mínima 1	`CHECK (quantidade >= 1)` em `item_carrinho`
RN05	Sem itens negativos	Mesmo `CHECK`; a remoção apaga a linha
RN06	Valores consistentes	`vw_carrinho_detalhado` calcula subtotal e total no banco
RN07	Endereço obrigatório	Colunas `NOT NULL`, domínios `d_cep` e `d_uf`; `fn_confirmar_pedido` exige endereço
RN08	Métodos válidos	Tipo enumerado `metodo_pagamento` (PIX, CARTAO)
RN09	Pedido válido	`fn_confirmar_pedido` valida carrinho, estoque e produtos ativos
RN10	Confirmar pagamento	Trigger que muda o pedido para PAGO quando o pagamento é APROVADO
RN11	Status válido	Tipo enumerado `status_pedido`
RN12	Campos obrigatórios	`NOT NULL`, `CHECK (preco > 0)` e `UNIQUE (nome)`
RN13	Produto existente	Edição por chave primária; produtos com pedidos não são apagados (`ON DELETE RESTRICT`)
RN14	Fluxo correto	Tabela `transicao_status_pedido` e trigger de validação
RN15	Dados atualizados	Views calculadas na consulta, sem tabela de cópia

Tabela 3: Regras de negócio e como o banco as garante

1.6 Dicionário de dados

Legenda: PK = chave primária; FK = chave estrangeira; UK = valor único. As colunas de data e hora usam `TIMESTAMPTZ`.

1.6.1 Tabela `usuario`

Generalização de Cliente e Administrador; guarda credenciais de acesso.

Coluna	Tipo	Chave	Nulo?	Descrição
id	BIGINT	PK	Não	Identificador do usuário
nome	VARCHAR(100)	–	Não	Nome completo (mín. 2 caracteres)
email	VARCHAR(150)	UK	Não	E-mail de login (único, formato validado)
senha_hash	VARCHAR(255)	–	Não	Hash da senha (nunca texto puro)
perfil	perfil_usuario	–	Não	CLIENTE ou ADMINISTRADOR
ativo	BOOLEAN	–	Não	Indica se a conta está ativa
criado_em	TIMESTAMPTZ	–	Não	Data/hora de criação

1.6.2 Tabela `cliente`

Especialização 1:1 de Usuario com dados exclusivos do cliente.

Coluna	Tipo	Chave	Nulo?	Descrição
usuario_id	BIGINT	PK, FK	Não	Identificador do cliente (= usuario.id) (ref. usuario)
telefone	VARCHAR(15)	–	Sim	Telefone apenas com dígitos (10 a 15)

1.6.3 Tabela `endereco`

Endereços de entrega salvos pelo cliente (US07).

Coluna	Tipo	Chave	Nulo?	Descrição
id	BIGINT	PK	Não	Identificador do endereço
cliente_id	BIGINT	FK	Não	Cliente proprietário (ref. cliente)
apelido	VARCHAR(30)	–	Sim	Nome amigável (ex.: Casa)
cep	d_cep	–	Não	CEP com 8 dígitos
logradouro	VARCHAR(150)	–	Não	Rua/avenida
numero	VARCHAR(10)	–	Não	Número do imóvel
complemento	VARCHAR(60)	–	Sim	Complemento (opcional)
bairro	VARCHAR(80)	–	Não	Bairro
cidade	VARCHAR(80)	–	Não	Cidade
uf	d_uf	–	Não	Sigla do estado (2 letras maiúsculas)
principal	BOOLEAN	–	Não	Endereço padrão (no máximo um ativo por cliente)
ativo	BOOLEAN	–	Não	Exclusão lógica do endereço

1.6.4 Tabela `categoria`

Classificação dos cupcakes para filtro (US02).

Coluna	Tipo	Chave	Nulo?	Descrição
id	SMALLINT	PK	Não	Identificador da categoria
nome	VARCHAR(50)	UK	Não	Nome único (ex.: Vegano)
ativa	BOOLEAN	–	Não	Categoria disponível para filtro

1.6.5 Tabela `cupcake`

Produto vendido na loja (US01, US03, US12, US13).

Coluna	Tipo	Chave	Nulo?	Descrição
`id`	BIGINT	PK	Não	Identificador do produto
`nome`	VARCHAR(100)	UK	Não	Nome único do cupcake
`descricao`	TEXT	–	Não	Descrição exibida ao cliente
`ingredientes`	TEXT	–	Sim	Lista de ingredientes
`preco`	NUMERIC(10,2)	–	Não	Preço atual de venda (> 0)
`imagem_url`	VARCHAR(255)	–	Sim	Endereço da imagem
`estoque`	INTEGER	–	Não	Unidades disponíveis (≥ 0)
`destaque`	BOOLEAN	–	Não	Ordena a vitrine (CA03 da US01)
`ativo`	BOOLEAN	–	Não	RN01: somente ativos aparecem na vitrine
`criado_em`	TIMESTAMPTZ	–	Não	Data/hora de cadastro
`atualizado_em`	TIMESTAMPTZ	–	Não	Última alteração (trigger)

1.6.6 Tabela `cupcake_categoria`

Associação N:N entre cupcake e categoria.

Coluna	Tipo	Chave	Nulo?	Descrição
`cupcake_id`	BIGINT	PK, FK	Não	Cupcake classificado (ref. cupcake)
`categoria_id`	SMALLINT	PK, FK	Não	Categoria atribuída (ref. categoria)

1.6.7 Tabela `carrinho`

Carrinho persistido no banco (US04 a US06, RNF04).

Coluna	Tipo	Chave	Nulo?	Descrição
`id`	BIGINT	PK	Não	Identificador do carrinho
`cliente_id`	BIGINT	FK	Não	Dono do carrinho (ref. cliente)
`status`	status_carrinho	–	Não	ABERTO, CONVERTIDO ou ABANDONADO (um ABERTO por cliente)
`criado_em`	TIMESTAMPTZ	–	Não	Criação
`atualizado_em`	TIMESTAMPTZ	–	Não	Última alteração (trigger)

1.6.8 Tabela `item_carrinho`

Itens do carrinho (classe ItemCarrinho).

Coluna	Tipo	Chave	Nulo?	Descrição
`carrinho_id`	BIGINT	PK, FK	Não	Carrinho (ref. carrinho)
`cupcake_id`	BIGINT	PK, FK	Não	Produto (ref. cupcake)
`quantidade`	SMALLINT	–	Não	RN04/RN05: mínimo 1

1.6.9 Tabela `pedido`

Pedido confirmado (US09); guarda cópia do endereço de entrega.

Coluna	Tipo	Chave	Nulo?	Descrição
`id`	BIGINT	PK	Não	Número do pedido
`cliente_id`	BIGINT	FK	Não	Cliente que comprou (ref. cliente)
`status`	status_pedido	–	Não	Situação atual (fluxo controlado por RN14)
`data_pedido`	TIMESTAMPTZ	–	Não	Data/hora da confirmação
`entrega_cep`	d_cep	–	Não	CEP de entrega (cópia)
`entrega_logradouro`	VARCHAR(150)	–	Não	Rua de entrega (cópia)
`entrega_numero`	VARCHAR(10)	–	Não	Número de entrega (cópia)
`entrega_complemento`	VARCHAR(60)	–	Sim	Complemento (cópia)
`entrega_bairro`	VARCHAR(80)	–	Não	Bairro de entrega (cópia)
`entrega_cidade`	VARCHAR(80)	–	Não	Cidade de entrega (cópia)
`entrega_uf`	d_uf	–	Não	UF de entrega (cópia)
`taxa_entrega`	NUMERIC(10,2)	–	Não	Taxa de entrega em R$
`valor_total`	NUMERIC(10,2)	–	Não	Itens + taxa de entrega em R$
`atualizado_em`	TIMESTAMPTZ	–	Não	Última alteração (trigger)

1.6.10 Tabela `item_pedido`

Itens do pedido com preço congelado.

Coluna	Tipo	Chave	Nulo?	Descrição
`pedido_id`	BIGINT	PK, FK	Não	Pedido (ref. pedido)
`cupcake_id`	BIGINT	PK, FK	Não	Produto vendido (ref. cupcake)
`quantidade`	SMALLINT	–	Não	Quantidade (≥ 1)
`preco_unitario`	NUMERIC(10,2)	–	Não	Preço no momento da compra
`subtotal`	NUMERIC(10,2)	–	Não	quantidade x preco_unitario (coluna gerada)

1.6.11 Tabela `pagamento`

Tentativas de pagamento de um pedido (US08, US10).

Coluna	Tipo	Chave	Nulo?	Descrição
`id`	BIGINT	PK	Não	Identificador da tentativa
`pedido_id`	BIGINT	FK	Não	Pedido pago (ref. pedido)
`metodo`	metodo_pagamento	–	Não	PIX ou CARTAO (RN08)
`status`	status_pagamento	–	Não	PENDENTE, APROVADO, RECUSADO ou ESTORNADO
`valor`	NUMERIC(10,2)	–	Não	Valor cobrado (> 0)
`gateway_transacao_id`	VARCHAR(100)	UK	Sim	ID da transação no gateway (único; sem dados de cartão)
`criado_em`	TIMESTAMPTZ	–	Não	Início da tentativa
`processado_em`	TIMESTAMPTZ	–	Sim	Retorno do gateway

1.6.12 Tabela `transicao_status_pedido`

Tabela de apoio com o fluxo válido de status (RN14).

Coluna	Tipo	Chave	Nulo?	Descrição
`de`	status_pedido	PK	Não	Status de origem
`para`	status_pedido	PK	Não	Status de destino permitido

1.6.13 Tabela `historico_status_pedido`

Auditoria das mudanças de status; alimenta o acompanhamento (US11).

Coluna	Tipo	Chave	Nulo?	Descrição
`id`	BIGINT	PK	Não	Identificador do registro
`pedido_id`	BIGINT	FK	Não	Pedido (ref. pedido)
`status_anterior`	status_pedido	–	Sim	Status antes da mudança (nulo na criação)
`status_novo`	status_pedido	–	Não	Status após a mudança
`alterado_por`	BIGINT	FK	Sim	Usuário responsável (ref. usuario)
`alterado_em`	TIMESTAMPTZ	–	Não	Data/hora da mudança

1.6.14 Tipos enumerados

Tipo	Valores
`perfil_usuario`	CLIENTE, ADMINISTRADOR
`status_carrinho`	ABERTO, CONVERTIDO, ABANDONADO
`status_pedido`	AGUARDANDO_PAGAMENTO, PAGO, EM_PREPARO, ENVIADO, ENTREGUE, CANCELADO
`metodo_pagamento`	PIX, CARTAO
`status_pagamento`	PENDENTE, APROVADO, RECUSADO, ESTORNADO
`d_cep`	CHAR(8), somente dígitos
`d_uf`	CHAR(2), letras maiúsculas
`d_dinheiro`	NUMERIC(10,2), valor ≥ 0

Tabela 17: Tipos enumerados e domínios

1.6.15 Fluxo permitido de status do pedido (RN14)

AGUARDANDO_PAGAMENTO → PAGO → EM_PREPARO → ENVIADO → ENTREGUE

Cancelamento (CANCELADO) é permitido a partir de AGUARDANDO_PAGAMENTO, PAGO e EM_PREPARO.

1.7 Verificação do projeto físico

O script foi executado em um PostgreSQL 16 sem erros, e os cenários abaixo foram testados, primeiro diretamente no banco e depois pela aplicação (API e telas), com os dados de carga inicial:

- vitrine com apenas produtos ativos e suas categorias, e filtro por categoria;
- conversão do carrinho em pedido com baixa de estoque, preço congelado e total correto (itens mais taxa de entrega);
- pagamento recusado seguido de nova tentativa aprovada, que muda o pedido para PAGO;
- rejeição de transição inválida de status (de PAGO direto para ENTREGUE);
- rejeição de quantidade zero, nome inválido, administrador cadastrado como cliente, e-mail ou nome de produto repetidos, e confirmação com carrinho vazio ou inexistente;
- relatórios de vendas diárias, produtos mais vendidos e estoque;
- bloqueio de `DELETE` em pedidos para o papel da aplicação;
- acesso negado (403) de cliente às rotas do dono e (401) de visitante às rotas protegidas.

2 5ª Entrega: Implementação do Sistema

Esta seção descreve como o sistema foi construído a partir dos artefatos das entregas anteriores. O passo a passo de instalação e uso está no documento separado *Manual de Uso*. Os diagramas UML revisados e o modelo conceitual estão no documento *Revisão da Modelagem UML e Projeto Conceitual de Dados*, e os testes automatizados (131 casos de back-end e banco e 31 de interface, todos aprovados) estão descritos no *Documento de Testes*.

2.1 Repositório

O código-fonte da implementação (padrão MVC em Java), o banco de dados, os testes automatizados e a documentação estão no GitHub:

`https://github.com/Fellp-coffww/Loja-Cupcake`

Pasta ou arquivo	Conteúdo
`src/loja/`	Back-end: `model`, `dao`, `controller` e `util` (MVC) e `Main.java`
`public/`	Front-end (View): páginas, scripts e estilos
`sql/`	Esquema, regras, *triggers*, *views* e dados iniciais do PostgreSQL
`test/` e `testes/`	Testes automatizados de back-end (JUnit) e de interface (Playwright)
`docs/`	Manual de uso, documento do banco de dados, revisão UML e modelo conceitual, documento de testes
`README.md`	Como instalar, executar e testar

Tabela 18: Organização do repositório

2.2 Tecnologias

- **Back-end:** Java 17 ou superior, sem frameworks. O servidor HTTP é o que já acompanha o JDK (`com.sun.net.httpserver`) e o acesso ao banco usa JDBC com o driver oficial do PostgreSQL 42.7.
- **Banco de dados:** PostgreSQL 16, conforme o projeto da seção anterior.
- **Front-end:** HTML, CSS e JavaScript puros, sem bibliotecas, servidos pelo próprio back-end.
- **Formato de troca de dados:** JSON, com conversor próprio mínimo.

2.3 Arquitetura MVC

Camada	Local	Responsabilidade
Model	`model/` e `dao/`	Registros (Cupcake, Pedido, Endereco etc.) e acesso ao banco por DAOs com *PreparedStatement*
View	`public/`	Telas `index.html` (cliente) e `admin.html` (dono), com JavaScript e CSS
Controller	`controller/`	Recebem as requisições de `/api/...`, validam os dados, chamam os DAOs e devolvem JSON
Apoio	`util/` e `Main.java`	Conexão, JSON, sessão, hash de senha, respostas HTTP e registro das rotas

Tabela 19: Organização do código em MVC

Fluxo de uma requisição: o navegador (View) chama `fetch('/api/...')`, o Controller valida e delega ao DAO, o DAO consulta o PostgreSQL e o resultado volta em JSON para a View.

As regras de negócio mais sensíveis ficam no banco (quantidade mínima, fluxo de status do pedido, baixa de estoque, histórico). O Controller só valida o formato dos dados e traduz os erros do banco em mensagens claras em português.

2.4 Telas e navegação

O mapa navegacional da 3ª entrega foi adaptado na implementação:

1. **Login:** ao abrir o endereço, é a única tela exibida, com as abas "Entrar" e "Criar conta".
2. **Loja:** após o login, aparece a vitrine com filtro por categoria e janela de detalhes do produto.
3. **Aba lateral:** painel deslizante à direita, com duas abas: "Carrinho" (itens, total e botão *Finalizar pedido*) e "Meus pedidos" (status e pagamento).
4. **Finalizar pedido:** janela para escolher ou cadastrar o endereço e confirmar.
5. **Pagamento:** janela com PIX ou cartão, aberta após a confirmação ou pelo botão *Pagar agora*.
6. **Painel do dono** (`admin.html`): dashboard, pedidos com troca de status, cadastro e edição de produtos. O administrador é levado a esse painel ao fazer login.

2.5 Interface de programação (API REST)

Método e rota	Acesso	Histórias
`GET /api/cupcakes[?categoria=ID]`	Público	US01, US02

Método e rota	Acesso	Histórias
`GET /api/cupcakes/{id}`	Público	US03
`GET /api/categorias`	Público	US02
`POST /api/cadastro, /api/login, /api/logout`	Público	Acesso
`GET /api/carrinho`	Cliente	US06
`POST /api/carrinho/itens`	Cliente	US04
`DELETE /api/carrinho/itens/{id}`	Cliente	US05
`GET, POST /api/enderecos`	Cliente	US07
`POST /api/pedidos`	Cliente	US09
`POST /api/pedidos/{id}/pagamento`	Cliente	US08, US10
`GET /api/pedidos`	Cliente	US11
`GET, POST /api/admin/cupcakes`	Dono	US12
`PUT /api/admin/cupcakes/{id}`	Dono	US13
`GET /api/admin/pedidos`	Dono	US14
`PUT /api/admin/pedidos/{id}/status`	Dono	US14
`GET /api/admin/dashboard`	Dono	US15

2.6 Autenticação e segurança

- O login devolve um *token* que o navegador envia no cabeçalho `Authorization`. As sessões ficam em memória no servidor.
- Cada rota exige o perfil correto: rotas do cliente só aceitam CLIENTE e rotas `/api/admin` só aceitam ADMINISTRADOR (resposta 403 nos demais casos).
- As senhas são guardadas apenas como *hash* SHA-256; em produção recomenda-se bcrypt ou argon2.
- Todas as consultas usam parâmetros (*PreparedStatement*), o que evita injeção de SQL. O front-end escapa o texto exibido para evitar injeção de HTML.
- Dados de cartão não são solicitados nem armazenados.

2.7 Rastreabilidade: histórias de usuário e implementação

História	Situação	Observação
US01 Vitrine	Completa	Somente produtos ativos, ordenados por destaque
US02 Filtro por categoria	Completa	Atualização imediata da lista
US03 Detalhes	Completa	Descrição, ingredientes, imagem e preço
US04 Adicionar ao carrinho	Parcial	Cada clique soma uma unidade; a API aceita quantidade, mas a tela ainda não tem seletor
US05 Remover item	Parcial	Remove o item sem pedir confirmação (prevista no CU05)
US06 Visualizar carrinho	Completa	Subtotal por item e total
US07 Endereço de entrega	Parcial	Validação e gravação funcionam; falta a consulta automática ao ViaCEP (T7.2)
US08 Forma de pagamento	Completa	PIX ou cartão

História	Situação	Observação
US09 Confirmar pedido	Parcial	Pedido criado em transação única; falta tela de revisão final (T9.1)
US10 Processar pagamento	Parcial	Gateway simulado, com opção de recusa para demonstração; sem *webhook* (T10.2)
US11 Acompanhar pedido	Parcial	Atualização periódica (15 s), sem tempo real
US12 Cadastrar produto	Completa	Imagem informada por URL
US13 Editar produto	Completa	Inclui ativar, desativar e ajustar estoque
US14 Atualizar status	Completa	Fluxo validado pelo banco (RN14)
US15 Dashboard de vendas	Parcial	Tabelas de vendas, mais vendidos e estoque; sem gráficos (T15.1)

2.8 Testes automatizados

O sistema tem testes automatizados em cinco níveis: unitário (conversor JSON, senha e sessão), integração dos DAOs com o PostgreSQL, regras do próprio banco, API de ponta a ponta e interface no navegador. Todos são repetíveis com os scripts `testar.bat` e `testar_interface.py`. Os testes encontraram dois defeitos de tratamento de entrada (JSON cortado e número decimal em campo inteiro), já corrigidos.

2.9 Simplificações e trabalhos futuros

- Integrar um gateway de pagamento real em modo de testes (*sandbox*) e tratar o *webhook*.
- Consultar o CEP no ViaCEP e preencher o endereço automaticamente.
- Seletor de quantidade no carrinho, tela de revisão antes de confirmar o pedido e gráficos no dashboard.
- Entrada de estoque com histórico de movimentações e devolução ao estoque em pedidos cancelados.

A Script SQL completo (PostgreSQL)

```sql
-- ============================================================================
-- Loja Virtual de Cupcakes - Projeto físico do banco de dados
-- SGBD: PostgreSQL 14+ (testado no 16)
-- Projeto Integrador Transdisciplinar (PITE) - Engenharia de Software
-- Autor: Fellipe Augusto Gonçalves Magalhães
-- ============================================================================

-- Recria o schema do zero (apenas para ambiente de desenvolvimento)
DROP SCHEMA IF EXISTS loja CASCADE;
CREATE SCHEMA loja;
SET search_path TO loja;

-- ----------------------------------------------------------------------------
-- 1. DOMÍNIOS E TIPOS ENUMERADOS
-- ----------------------------------------------------------------------------
CREATE TYPE perfil_usuario     AS ENUM ('CLIENTE', 'ADMINISTRADOR');
CREATE TYPE status_carrinho    AS ENUM ('ABERTO', 'CONVERTIDO', 'ABANDONADO');
CREATE TYPE status_pedido      AS ENUM ('AGUARDANDO_PAGAMENTO', 'PAGO', 'EM_PREPARO',
                                        'ENVIADO', 'ENTREGUE', 'CANCELADO');
CREATE TYPE metodo_pagamento   AS ENUM ('PIX', 'CARTAO');
CREATE TYPE status_pagamento   AS ENUM ('PENDENTE', 'APROVADO', 'RECUSADO', 'ESTORNADO');

CREATE DOMAIN d_cep AS CHAR(8)      CHECK (VALUE ~ '^[0-9]{8}$');
CREATE DOMAIN d_uf  AS CHAR(2)      CHECK (VALUE ~ '^[A-Z]{2}$');
CREATE DOMAIN d_dinheiro AS NUMERIC(10,2) CHECK (VALUE >= 0);

-- ----------------------------------------------------------------------------
-- 2. TABELAS
-- ----------------------------------------------------------------------------

-- Classe Usuario (generalização de Cliente e Administrador)
CREATE TABLE usuario (
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome           VARCHAR(100)  NOT NULL CHECK (length(trim(nome)) >= 2),
    email          VARCHAR(150)  NOT NULL,
    senha_hash     VARCHAR(255)  NOT NULL,          -- nunca armazenar senha em texto puro
    perfil         perfil_usuario NOT NULL DEFAULT 'CLIENTE',
    ativo          BOOLEAN       NOT NULL DEFAULT TRUE,
    criado_em      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_usuario_email UNIQUE (email),
    CONSTRAINT ck_usuario_email CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$')
);

-- Classe Cliente (especialização 1:1 de Usuario)
CREATE TABLE cliente (
    usuario_id     BIGINT PRIMARY KEY REFERENCES usuario(id) ON DELETE CASCADE,
    telefone       VARCHAR(15) CHECK (telefone ~ '^[0-9]{10,15}$')
);

-- Atributo "endereco: String" do diagrama de classes, normalizado
CREATE TABLE endereco (
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id     BIGINT       NOT NULL REFERENCES cliente(usuario_id) ON DELETE CASCADE,
    apelido        VARCHAR(30),                     -- ex.: "Casa", "Trabalho"
    cep            d_cep        NOT NULL,
    logradouro     VARCHAR(150) NOT NULL,
    numero         VARCHAR(10)  NOT NULL,
    complemento    VARCHAR(60),
    bairro         VARCHAR(80)  NOT NULL,
    cidade         VARCHAR(80)  NOT NULL,
    uf             d_uf         NOT NULL,
    principal      BOOLEAN      NOT NULL DEFAULT FALSE,
    ativo          BOOLEAN      NOT NULL DEFAULT TRUE
);
-- no máximo um endereço principal ativo por cliente
CREATE UNIQUE INDEX uq_endereco_principal
    ON endereco (cliente_id) WHERE principal AND ativo;

CREATE TABLE categoria (
    id             SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome           VARCHAR(50) NOT NULL,
    ativa          BOOLEAN     NOT NULL DEFAULT TRUE,
    CONSTRAINT uq_categoria_nome UNIQUE (nome)
);

CREATE TABLE cupcake (
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome           VARCHAR(100) NOT NULL,
    descricao      TEXT         NOT NULL,
    ingredientes   TEXT,
    preco          d_dinheiro   NOT NULL CHECK (preco > 0),
    imagem_url     VARCHAR(255),
    estoque        INTEGER      NOT NULL DEFAULT 0 CHECK (estoque >= 0),
    destaque       BOOLEAN      NOT NULL DEFAULT FALSE,
    ativo          BOOLEAN      NOT NULL DEFAULT TRUE,   -- RN01: exibir só ativos
    criado_em      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    atualizado_em  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT uq_cupcake_nome UNIQUE (nome)
);

-- Relação N:N: um cupcake pode ser "chocolate" e "vegano" ao mesmo tempo (US02)
CREATE TABLE cupcake_categoria (
    cupcake_id     BIGINT   NOT NULL REFERENCES cupcake(id)   ON DELETE CASCADE,
    categoria_id   SMALLINT NOT NULL REFERENCES categoria(id) ON DELETE RESTRICT,
    PRIMARY KEY (cupcake_id, categoria_id)
);
```

```
97
98 -- Classe Carrinho (persistido no banco: RNF04)
99 CREATE TABLE carrinho (
100     id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
101     cliente_id      BIGINT          NOT NULL REFERENCES cliente(usuario_id) ON DELETE CASCADE,
102     status          status_carrinho NOT NULL DEFAULT 'ABERTO',
103     criado_em       TIMESTAMPTZ     NOT NULL DEFAULT now(),
104     atualizado_em   TIMESTAMPTZ     NOT NULL DEFAULT now()
105 );
106 -- cada cliente tem no máximo 1 carrinho aberto
107 CREATE UNIQUE INDEX uq_carrinho_aberto ON carrinho (cliente_id) WHERE status = 'ABERTO';
108
109 CREATE TABLE item_carrinho (
110     carrinho_id     BIGINT   NOT NULL REFERENCES carrinho(id) ON DELETE CASCADE,
111     cupcake_id      BIGINT   NOT NULL REFERENCES cupcake(id)  ON DELETE RESTRICT,
112     quantidade      SMALLINT NOT NULL CHECK (quantidade >= 1),   -- RN04 / RN05
113     PRIMARY KEY (carrinho_id, cupcake_id)
114 );
115
116 -- Classe Pedido. O endereço é "congelado" (snapshot) no momento da compra.
117 CREATE TABLE pedido (
118     id                  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
119     cliente_id          BIGINT        NOT NULL REFERENCES cliente(usuario_id) ON DELETE RESTRICT,
120     status              status_pedido NOT NULL DEFAULT 'AGUARDANDO_PAGAMENTO',
121     data_pedido         TIMESTAMPTZ   NOT NULL DEFAULT now(),
122     entrega_cep         d_cep         NOT NULL,
123     entrega_logradouro VARCHAR(150)  NOT NULL,
124     entrega_numero      VARCHAR(10)   NOT NULL,
125     entrega_complemento VARCHAR(60),
126     entrega_bairro      VARCHAR(80)   NOT NULL,
127     entrega_cidade      VARCHAR(80)   NOT NULL,
128     entrega_uf          d_uf          NOT NULL,
129     taxa_entrega        d_dinheiro    NOT NULL DEFAULT 0,
130     valor_total         d_dinheiro    NOT NULL,
131     atualizado_em       TIMESTAMPTZ   NOT NULL DEFAULT now()
132 );
133
134 -- Preço unitário gravado no pedido: alterar o preço do cupcake não muda pedidos antigos
135 CREATE TABLE item_pedido (
136     pedido_id       BIGINT     NOT NULL REFERENCES pedido(id)  ON DELETE CASCADE,
137     cupcake_id      BIGINT     NOT NULL REFERENCES cupcake(id) ON DELETE RESTRICT,
138     quantidade      SMALLINT   NOT NULL CHECK (quantidade >= 1),
139     preco_unitario d_dinheiro NOT NULL CHECK (preco_unitario > 0),
140     subtotal        NUMERIC(10,2) GENERATED ALWAYS AS (quantidade * preco_unitario) STORED,
141     PRIMARY KEY (pedido_id, cupcake_id)
142 );
143
144 -- Classe Pagamento. 1 pedido -> N tentativas (CU10: pagamento recusado volta à escolha)
145 -- Dados de cartão NÃO são armazenados (RNF08): apenas o identificador do gateway.
146 CREATE TABLE pagamento (
147     id                   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
148     pedido_id            BIGINT          NOT NULL REFERENCES pedido(id) ON DELETE RESTRICT,
149     metodo               metodo_pagamento NOT NULL,
150     status               status_pagamento NOT NULL DEFAULT 'PENDENTE',
151     valor                d_dinheiro       NOT NULL CHECK (valor > 0),
152     gateway_transacao_id VARCHAR(100),
153     criado_em            TIMESTAMPTZ      NOT NULL DEFAULT now(),
154     processado_em        TIMESTAMPTZ,
155     CONSTRAINT uq_pagamento_gateway UNIQUE (gateway_transacao_id),
156     CONSTRAINT ck_pagamento_processado
157         CHECK (status = 'PENDENTE' OR processado_em IS NOT NULL)
158 );
159 -- no máximo um pagamento APROVADO por pedido
160 CREATE UNIQUE INDEX uq_pagamento_aprovado ON pagamento (pedido_id) WHERE status = 'APROVADO';
161
162 -- Fluxo permitido de status (RN14 - "fluxo correto") e histórico (auditoria / US11)
163 CREATE TABLE transicao_status_pedido (
164     de              status_pedido NOT NULL,
165     para            status_pedido NOT NULL,
166     PRIMARY KEY (de, para)
167 );
168 INSERT INTO transicao_status_pedido (de, para) VALUES
169     ('AGUARDANDO_PAGAMENTO', 'PAGO'),
170     ('AGUARDANDO_PAGAMENTO', 'CANCELADO'),
171     ('PAGO',                 'EM_PREPARO'),
172     ('PAGO',                 'CANCELADO'),
173     ('EM_PREPARO',           'ENVIADO'),
174     ('EM_PREPARO',           'CANCELADO'),
175     ('ENVIADO',              'ENTREGUE');
176
177 CREATE TABLE historico_status_pedido (
178     id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
179     pedido_id       BIGINT        NOT NULL REFERENCES pedido(id) ON DELETE CASCADE,
180     status_anterior status_pedido,
181     status_novo     status_pedido NOT NULL,
182     alterado_por    BIGINT        REFERENCES usuario(id) ON DELETE SET NULL,
183     alterado_em     TIMESTAMPTZ   NOT NULL DEFAULT now()
184 );
185
186 -- ---------------------------------------------------------------------------
187 -- 3. ÍNDICES DE DESEMPENHO (RNF01, RNF02, RNF15)
188 -- ---------------------------------------------------------------------------
189 CREATE INDEX ix_cupcake_vitrine      ON cupcake (destaque DESC, nome) WHERE ativo;
190 CREATE INDEX ix_cupcake_categoria    ON cupcake_categoria (categoria_id, cupcake_id);
191 CREATE INDEX ix_endereco_cliente     ON endereco (cliente_id);
192 CREATE INDEX ix_pedido_cliente       ON pedido (cliente_id, data_pedido DESC);
193 CREATE INDEX ix_pedido_status_data   ON pedido (status, data_pedido);
194 CREATE INDEX ix_item_pedido_cupcake  ON item_pedido (cupcake_id);
195 CREATE INDEX ix_pagamento_pedido     ON pagamento (pedido_id);
```

```
196 CREATE INDEX ix_historico_pedido     ON historico_status_pedido (pedido_id, alterado_em);
197
198 -- ------------------------------------------------------------------------
199 -- 4. TRIGGERS (regras de negócio no banco)
200 -- ------------------------------------------------------------------------
201
202 -- 4.1 Atualiza "atualizado_em" automaticamente
203 CREATE FUNCTION fn_toca_atualizado_em() RETURNS trigger LANGUAGE plpgsql AS $$
204 BEGIN
205     NEW.atualizado_em := now();
206     RETURN NEW;
207 END $$;
208
209 CREATE TRIGGER tg_cupcake_atualizado  BEFORE UPDATE ON cupcake
210     FOR EACH ROW EXECUTE FUNCTION fn_toca_atualizado_em();
211 CREATE TRIGGER tg_carrinho_atualizado BEFORE UPDATE ON carrinho
212     FOR EACH ROW EXECUTE FUNCTION fn_toca_atualizado_em();
213 CREATE TRIGGER tg_pedido_atualizado   BEFORE UPDATE ON pedido
214     FOR EACH ROW EXECUTE FUNCTION fn_toca_atualizado_em();
215
216 -- 4.2 RN14: valida a transição de status e grava histórico
217 CREATE FUNCTION fn_valida_status_pedido() RETURNS trigger LANGUAGE plpgsql AS $$
218 BEGIN
219     IF NEW.status IS DISTINCT FROM OLD.status THEN
220         IF NOT EXISTS (SELECT 1 FROM transicao_status_pedido
221                        WHERE de = OLD.status AND para = NEW.status) THEN
222             RAISE EXCEPTION 'Transição de status inválida: % -> %', OLD.status, NEW.status
223                 USING ERRCODE = 'check_violation';
224         END IF;
225         INSERT INTO historico_status_pedido (pedido_id, status_anterior, status_novo, alterado_por)
226         VALUES (NEW.id, OLD.status, NEW.status,
227                 NULLIF(current_setting('loja.usuario_id', true), '')::BIGINT);
228     END IF;
229     RETURN NEW;
230 END $$;
231
232 CREATE TRIGGER tg_pedido_status BEFORE UPDATE OF status ON pedido
233     FOR EACH ROW EXECUTE FUNCTION fn_valida_status_pedido();
234
235 -- 4.3 Registra o status inicial do pedido no histórico
236 CREATE FUNCTION fn_historico_inicial() RETURNS trigger LANGUAGE plpgsql AS $$
237 BEGIN
238     INSERT INTO historico_status_pedido (pedido_id, status_anterior, status_novo, alterado_por)
239     VALUES (NEW.id, NULL, NEW.status, NEW.cliente_id);
240     RETURN NEW;
241 END $$;
242
243 CREATE TRIGGER tg_pedido_historico_inicial AFTER INSERT ON pedido
244     FOR EACH ROW EXECUTE FUNCTION fn_historico_inicial();
245
246 -- 4.4 Pagamento aprovado => pedido passa para PAGO (US10 / RN10)
247 CREATE FUNCTION fn_pagamento_aprovado() RETURNS trigger LANGUAGE plpgsql AS $$
248 BEGIN
249     IF NEW.status = 'APROVADO' AND OLD.status IS DISTINCT FROM 'APROVADO' THEN
250         UPDATE pedido SET status = 'PAGO'
251          WHERE id = NEW.pedido_id AND status = 'AGUARDANDO_PAGAMENTO';
252     END IF;
253     RETURN NEW;
254 END $$;
255
256 CREATE TRIGGER tg_pagamento_aprovado AFTER UPDATE OF status ON pagamento
257     FOR EACH ROW EXECUTE FUNCTION fn_pagamento_aprovado();
258
259 -- 4.5 Só administradores podem ter o perfil usado para alterar produtos/pedidos
260 --     (garantia mínima no banco: cliente_id só pode referenciar usuário CLIENTE)
261 CREATE FUNCTION fn_cliente_deve_ser_perfil_cliente() RETURNS trigger LANGUAGE plpgsql AS $$
262 BEGIN
263     IF NOT EXISTS (SELECT 1 FROM usuario WHERE id = NEW.usuario_id AND perfil = 'CLIENTE') THEN
264         RAISE EXCEPTION 'Usuário % não possui perfil CLIENTE', NEW.usuario_id
265             USING ERRCODE = 'check_violation';
266     END IF;
267     RETURN NEW;
268 END $$;
269
270 CREATE TRIGGER tg_cliente_perfil BEFORE INSERT OR UPDATE ON cliente
271     FOR EACH ROW EXECUTE FUNCTION fn_cliente_deve_ser_perfil_cliente();
272
273 -- ------------------------------------------------------------------------
274 -- 5. FUNÇÃO TRANSACIONAL: CONFIRMAR PEDIDO (CU09 / US09)
275 --    Converte o carrinho aberto do cliente em pedido, copia preços e endereço,
276 --    baixa o estoque e fecha o carrinho. Tudo ou nada.
277 -- ------------------------------------------------------------------------
278 CREATE FUNCTION fn_confirmar_pedido(p_cliente_id BIGINT,
279                                     p_endereco_id BIGINT,
280                                     p_taxa_entrega NUMERIC DEFAULT 0)
281 RETURNS BIGINT LANGUAGE plpgsql AS $$
282 DECLARE
283     v_carrinho_id BIGINT;
284     v_pedido_id   BIGINT;
285     v_total       NUMERIC(10,2);
286 BEGIN
287     SELECT id INTO v_carrinho_id
288       FROM carrinho WHERE cliente_id = p_cliente_id AND status = 'ABERTO' FOR UPDATE;
289     IF v_carrinho_id IS NULL THEN
290         RAISE EXCEPTION 'Cliente % não possui carrinho aberto', p_cliente_id;
291     END IF;
292
293     IF NOT EXISTS (SELECT 1 FROM item_carrinho WHERE carrinho_id = v_carrinho_id) THEN
294         RAISE EXCEPTION 'Carrinho vazio';
```

```
295     END IF;
296
297     -- bloqueia os cupcakes e valida disponibilidade (RN09)
298     PERFORM 1 FROM cupcake c
299       JOIN item_carrinho i ON i.cupcake_id = c.id
300      WHERE i.carrinho_id = v_carrinho_id FOR UPDATE OF c;
301
302     IF EXISTS (SELECT 1 FROM item_carrinho i JOIN cupcake c ON c.id = i.cupcake_id
303                 WHERE i.carrinho_id = v_carrinho_id
304                   AND (NOT c.ativo OR c.estoque < i.quantidade)) THEN
305         RAISE EXCEPTION 'Há item inativo ou sem estoque suficiente no carrinho';
306     END IF;
307
308     SELECT SUM(i.quantidade * c.preco) INTO v_total
309       FROM item_carrinho i JOIN cupcake c ON c.id = i.cupcake_id
310      WHERE i.carrinho_id = v_carrinho_id;
311
312     INSERT INTO pedido (cliente_id, entrega_cep, entrega_logradouro, entrega_numero,
313                         entrega_complemento, entrega_bairro, entrega_cidade, entrega_uf,
314                         taxa_entrega, valor_total)
315     SELECT e.cliente_id, e.cep, e.logradouro, e.numero, e.complemento,
316            e.bairro, e.cidade, e.uf, p_taxa_entrega, v_total + p_taxa_entrega
317       FROM endereco e
318      WHERE e.id = p_endereco_id AND e.cliente_id = p_cliente_id AND e.ativo
319     RETURNING id INTO v_pedido_id;
320
321     IF v_pedido_id IS NULL THEN
322         RAISE EXCEPTION 'Endereço % inválido para o cliente %', p_endereco_id, p_cliente_id;
323     END IF;
324
325     INSERT INTO item_pedido (pedido_id, cupcake_id, quantidade, preco_unitario)
326     SELECT v_pedido_id, i.cupcake_id, i.quantidade, c.preco
327       FROM item_carrinho i JOIN cupcake c ON c.id = i.cupcake_id
328      WHERE i.carrinho_id = v_carrinho_id;
329
330     UPDATE cupcake c SET estoque = c.estoque - i.quantidade
331       FROM item_carrinho i
332      WHERE i.carrinho_id = v_carrinho_id AND c.id = i.cupcake_id;
333
334     UPDATE carrinho SET status = 'CONVERTIDO' WHERE id = v_carrinho_id;
335
336     RETURN v_pedido_id;
337 END $$;
338
339 -- ============================================================================
340 -- 6. VIEWS (consultas prontas para a API e para o dashboard - US01, US15)
341 --    A classe "Relatorio" do diagrama não vira tabela: é gerada por consulta.
342 -- ============================================================================
343
344 -- Vitrine (RN01: somente ativos; ordenada por destaque)
345 CREATE VIEW vw_vitrine AS
346 SELECT c.id, c.nome, c.preco, c.imagem_url, c.destaque,
347        COALESCE(string_agg(cat.nome, ', ' ORDER BY cat.nome), '') AS categorias
348   FROM cupcake c
349   LEFT JOIN cupcake_categoria cc ON cc.cupcake_id = c.id
350   LEFT JOIN categoria cat        ON cat.id = cc.categoria_id AND cat.ativa
351  WHERE c.ativo
352  GROUP BY c.id
353  ORDER BY c.destaque DESC, c.nome;
354
355 -- Carrinho com subtotais e total (US06)
356 CREATE VIEW vw_carrinho_detalhado AS
357 SELECT ca.id AS carrinho_id, ca.cliente_id, c.id AS cupcake_id, c.nome,
358        i.quantidade, c.preco, (i.quantidade * c.preco) AS subtotal,
359        SUM(i.quantidade * c.preco) OVER (PARTITION BY ca.id) AS total
360   FROM carrinho ca
361   JOIN item_carrinho i ON i.carrinho_id = ca.id
362   JOIN cupcake c       ON c.id = i.cupcake_id
363  WHERE ca.status = 'ABERTO';
364
365 -- Vendas por dia (considera pedidos pagos ou além, exceto cancelados)
366 CREATE VIEW vw_vendas_diarias AS
367 SELECT date_trunc('day', data_pedido)::date AS dia,
368        COUNT(*)                  AS qtd_pedidos,
369        SUM(valor_total)          AS faturamento,
370        ROUND(AVG(valor_total),2) AS ticket_medio
371   FROM pedido
372  WHERE status NOT IN ('AGUARDANDO_PAGAMENTO', 'CANCELADO')
373  GROUP BY 1
374  ORDER BY 1 DESC;
375
376 -- Ranking de produtos mais vendidos
377 CREATE VIEW vw_produtos_mais_vendidos AS
378 SELECT c.id, c.nome, SUM(ip.quantidade) AS unidades_vendidas, SUM(ip.subtotal) AS receita
379   FROM item_pedido ip
380   JOIN pedido p  ON p.id = ip.pedido_id
381   JOIN cupcake c ON c.id = ip.cupcake_id
382  WHERE p.status NOT IN ('AGUARDANDO_PAGAMENTO', 'CANCELADO')
383  GROUP BY c.id, c.nome
384  ORDER BY unidades_vendidas DESC;
385
386 -- Relatório de estoque (gerarEstoque())
387 CREATE VIEW vw_estoque AS
388 SELECT id, nome, estoque, ativo,
389        CASE WHEN estoque = 0 THEN 'ESGOTADO'
390             WHEN estoque <= 5 THEN 'BAIXO'
391             ELSE 'OK' END AS situacao
392   FROM cupcake
393  ORDER BY estoque, nome;
```

```
394
395 -- Pedidos para acompanhamento (US11, US14)
396 CREATE VIEW vw_pedidos_resumo AS
397 SELECT p.id, p.cliente_id, u.nome AS cliente, p.status, p.data_pedido, p.valor_total,
398        (SELECT pg.status FROM pagamento pg WHERE pg.pedido_id = p.id
399          ORDER BY pg.criado_em DESC LIMIT 1) AS status_pagamento
400   FROM pedido p JOIN usuario u ON u.id = p.cliente_id;
401
402 -- ----------------------------------------------------------------------------
403 -- 7. PAPÉIS E PERMISSÕES (princípio do menor privilégio - RNF08, RNF12)
404 -- ----------------------------------------------------------------------------
405 DO $$
406 BEGIN
407     IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'app_loja') THEN
408         CREATE ROLE app_loja NOLOGIN;
409     END IF;
410 END $$;
411 GRANT USAGE ON SCHEMA loja TO app_loja;
412 GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA loja TO app_loja;
413 GRANT USAGE ON ALL SEQUENCES IN SCHEMA loja TO app_loja;
414 REVOKE DELETE ON pedido, item_pedido, pagamento, historico_status_pedido FROM app_loja;
415 -- pedidos e pagamentos nunca são apagados: apenas cancelados/estornados
416
417 -- ----------------------------------------------------------------------------
418 -- 8. DADOS INICIAIS (carga para desenvolvimento e testes)
419 -- ----------------------------------------------------------------------------
420 INSERT INTO usuario (nome, email, senha_hash, perfil) VALUES
421     ('Dona da Loja', 'admin@cupcakes.com', encode(sha256('123456'::bytea),'hex'), 'ADMINISTRADOR'),
422     ('Maria Silva',  'maria@email.com',    encode(sha256('123456'::bytea),'hex'), 'CLIENTE'),
423     ('João Souza',   'joao@email.com',     encode(sha256('123456'::bytea),'hex'),  'CLIENTE');
424
425 INSERT INTO cliente (usuario_id, telefone)
426     SELECT id, '62999990000' FROM usuario WHERE email = 'maria@email.com';
427 INSERT INTO cliente (usuario_id, telefone)
428     SELECT id, '62988880000' FROM usuario WHERE email = 'joao@email.com';
429
430 INSERT INTO endereco (cliente_id, apelido, cep, logradouro, numero, bairro, cidade, uf, principal)
431     SELECT id, 'Casa', '74000000', 'Rua das Flores', '123', 'Centro', 'Goiânia', 'GO', TRUE
432       FROM usuario WHERE email = 'maria@email.com';
433
434 INSERT INTO categoria (nome) VALUES ('Chocolate'), ('Vegano'), ('Sem lactose'), ('Frutas'), ('Tradicional');
435
436 INSERT INTO cupcake (nome, descricao, ingredientes, preco, estoque, destaque) VALUES
437     ('Brigadeiro Gourmet', 'Massa de chocolate com cobertura de brigadeiro', 'Farinha, cacau, ovos, leite condensado, granulado', 9.50, 30, TRUE),
438     ('Vegano de Cacau',    'Cupcake vegano de cacau com ganache',            'Farinha, cacau 70%, óleo de coco, leite vegetal', 11.00, 20, TRUE),
439     ('Morango Cremoso',    'Massa branca com creme de morango',              'Farinha, ovos, morango, creme de leite',          10.00, 15, FALSE),
440     ('Baunilha Clássico',  'Massa de baunilha com buttercream',              'Farinha, ovos, baunilha, manteiga',                8.00, 40, FALSE),
441     ('Limão sem Lactose',  'Massa cítrica sem lactose',                      'Farinha, limão, ovos, óleo vegetal',               9.00,  4, FALSE);
442
443 INSERT INTO cupcake_categoria (cupcake_id, categoria_id)
444     SELECT c.id, k.id FROM (VALUES
445         ('Brigadeiro Gourmet','Chocolate'),
446         ('Vegano de Cacau','Chocolate'), ('Vegano de Cacau','Vegano'), ('Vegano de Cacau','Sem lactose'),
447         ('Morango Cremoso','Frutas'),
448         ('Baunilha Clássico','Tradicional'),
449         ('Limão sem Lactose','Frutas'), ('Limão sem Lactose','Sem lactose')
450     ) AS v(cup, cat)
451     JOIN cupcake c ON c.nome = v.cup
452     JOIN categoria k ON k.nome = v.cat;
```